

```
!pip install gensim
```

```
Collecting gensim
```

```
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
```

```
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
```

```
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
```

```
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.1)
```

```
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.2)
```

```
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
```

```
----- 27.9/27.9 MB 32.0 MB/s eta 0:00:00
```

```
Installing collected packages: gensim
```

```
Successfully installed gensim-4.4.0
```

```
import re
```

```
import numpy as np
```

```
from sklearn.datasets import fetch_20newsgroups
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import classification_report, accuracy_score
```

```
from gensim.models import KeyedVectors
```

KeyedVectors koristimo u praksi kada radimo s već naučenim modelima (npr. Google News Word2Vec). Ne treniramo model, nego direktno koristimo gotove vektore riječi za zadatke poput klasifikacije, pretrage sličnosti ili analize teksta.

Uvozimo dataset fetch_20newsgroup i radit ćemo klasifikaciju članaka koristeći logističku regresiju

```
# -----  
# 1. Učitaj manji skup podataka  
# -----  
categories = [  
    "rec.sport.baseball",  
    "sci.space",  
    "talk.politics.mideast",
```

```
"comp.graphics"
]

dataset = fetch_20newsgroups(
    subset="train",
    categories=categories,
    remove=("headers", "footers", "quotes")
)

texts = dataset.data
y = dataset.target
target_names = dataset.target_names

print("Broj dokumenata:", len(texts))
print("Klase:", target_names)
```

```
Broj dokumenata: 2338
Klase: ['comp.graphics', 'rec.sport.baseball', 'sci.space', 'talk.politics.mideast']
```

```
# -----
# 2. Vrlo jednostavna tokenizacija
# -----
def tokenize(text: str) -> list[str]:
    text = text.lower()
    text = re.sub(r"^[a-z\s]", " ", text)
    return text.split()
```

✓ Korištenjem Regex biblioteke (re) radimo vrlo jednostavno čišćenje i tokenizaciju naših podataka

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# -----
# 3. Učitaj gotovi Word2Vec model
# -----
#
# Taj model ima 300-dimenzionalne vektore, pa će svaki dokument
# na kraju postati jedan vektor duljine 300.

model_path = "/content/drive/MyDrive/Colab Notebooks/GoogleNews-vectors-negative300.bin"
wv = KeyedVectors.load_word2vec_format(model_path, binary=True)

print("Dimenzija embeddinga:", wv.vector_size)
```

Dimenzija embeddinga: 300

- ✓ Učitavamo gotov Word2Vec model s Google Drivea koji ćemo koristiti za pretvaranje riječi u vektore. Model već sadrži naučene vektore riječi, pa nije potrebno trenirati model niti računati reprezentacije od početka.

```
# -----
# 4. Dokument -> prosjek word vectors
# -----
def document_vector(text: str, word_vectors: KeyedVectors) -> np.ndarray:
    tokens = tokenize(text)

    vectors = []
    for token in tokens:
        if token in word_vectors.key_to_index:
            vectors.append(word_vectors[token])

    if len(vectors) == 0:
        # Ako nijedna riječ nije u vocabularyju modela,
        # vrati nulti vektor.
        return np.zeros(word_vectors.vector_size, dtype=np.float32)

    return np.mean(vectors, axis=0)

X = np.array([document_vector(text, wv) for text in texts])
```

```
print("Shape matrice X:", X.shape)
```

```
Shape matrice X: (2338, 300)
```

- ✓ Za svaki token iz teksta (oni koji nisu u tekstu, njih ignoriramo) dohvaćamo odgovarajući vektor iz predtreniranog modela GoogleNews-vectors-negative300.bin.

Zatim računamo prosjek svih vektora riječi u članku kako bismo dobili jedan vektor koji predstavlja cijeli dokument, i taj vektor koristimo kao ulaz za klasifikaciju.

```
# -----  
# 5. Train/test split  
# -----  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)
```

```
# -----  
# 6. Klasifikator  
# -----  
clf = LogisticRegression(max_iter=2000)  
clf.fit(X_train, y_train)  
  
y_pred = clf.predict(X_test)  
  
print("Accuracy:", accuracy_score(y_test, y_pred))  
print()  
print(classification_report(y_test, y_pred, target_names=target_names))
```

```
Accuracy: 0.8653846153846154
```

```
precision    recall  f1-score   support
```

comp.graphics	0.88	0.83	0.85	117
rec.sport.baseball	0.82	0.94	0.88	119
sci.space	0.86	0.78	0.82	119
talk.politics.mideast	0.91	0.91	0.91	113
accuracy			0.87	468
macro avg	0.87	0.87	0.87	468
weighted avg	0.87	0.87	0.86	468

Počnite pisati kôd ili generirati uz pomoć umjetne inteligencije.